

DD_PAY-GW-01 - Payment Gateway Integration - Developer Implementation Guide

Version: 1.0 Status: Draft for review Bundle: 05_billing Audience: mid-level backend developers, integration developers, QA
Parent DD: DD_PAY-GW-01 - Payment Gateway Integration

1. How To Read This Guide

This guide is written in execution order. Follow it as the system runs, from provider setup, to callback receipt, to routing, to reconciliation and repair.

PAY-GW-01 does not apply invoice balances or wallet balances directly. It receives provider money notifications, stores them, then calls the correct internal module.

Target modules:

Payment target	Internal module	Internal API
POSTPAID invoice/account payment	BIL-01-PAY-01	POST /api/payments/receive
PREPAID wallet topup	BIL-05	POST /api/wallets/{wallet_id}/credit

2. Step 1 - Configure Provider And Channel

Before any provider callback is accepted, an admin or deployment seed must configure the provider and channel.

2.1 Seed Provider

Insert one provider row for M-Pesa Kenya:

```
insert into payment_gateway_provider (  
  provider_id,  
  operator_code,  
  provider_code,  
  provider_name,  
  provider_type,  
  status,  
  callback_url_path,  
  settlement_mode,  
  currency_code,  
  config_json,  
  secret_ref,  
  created_at,  
  updated_at  
) values (  
  'prv-mpesa-ke',  
  'WIK',  
  'MPESA_KE',  
  'M-Pesa Kenya',  
  'MOBILE_MONEY',  
  'ACTIVE',  
  '/api/payment-gateways/mpesa/callback',  
  'API',  
  'KES',  
  '{"businessShortCode":"123456","country":"KE"}',  
  'secret/paygw/mpesa-ke',  
  now(),  
  now()  
);
```

The secret_ref points to the secret manager entry that contains provider credentials. Do not store secrets in config_json.

2.2 Seed Channel

Create a channel for Paybill-style payments:

```
insert into payment_gateway_channel (  
  channel_id,  
  provider_id,  
  channel_code,
```

```

        payment_method,
        target_mode,
        status,
        display_name,
        display_instructions_json,
        routing_priority,
        metadata_json
    ) values (
        'chn-mpesa-paybill',
        'prv-mpesa-ke',
        'MPESA_PAYBILL',
        'MOBILE_MONEY',
        'AUTO_ROUTE',
        'ACTIVE',
        'M-Pesa Paybill',
        '{"en":"Use Paybill 123456 and enter your customer account number as the reference."}',
        10,
        '{"businessShortCode":"123456"}'
    );

```

Frontend apps use this channel row to show payment instructions. They should not hardcode M-Pesa text.

3. Step 2 - Provider Sends Callback

The provider sends a callback to:

```

POST /api/payment-gateways/mpesa/callback
Content-Type: application/json
X-Provider-Signature: sha256=example

```

Example request:

```

{
  "TransactionType": "Pay Bill",
  "TransID": "QL91ABCD",
  "TransTime": "20260529094200",
  "TransAmount": "2500.00",
  "BusinessShortCode": "123456",
  "BillRefNumber": "ACC-900045",
  "MSISDN": "254700111222",
  "FirstName": "Amina",
  "MiddleName": "",
  "LastName": "Otieno"
}

```

3.1 Controller Logic

The controller must do these actions in this order:

1. Find provider by callback route and operator context.
2. Verify signature or provider authentication.
3. Parse provider payload into normalized structure.
4. Build idempotency key.
5. Store callback event.
6. Upsert gateway transaction.
7. Return provider acknowledgement after durable store.

Pseudo-code:

```

public function mpesaCallback(Request $request): JsonResponse
{
    $provider = $this->providers->findActiveByCode('WIK', 'MPESA_KE');
    $adapter = $this->adapterRegistry->forProvider($provider);

    $signatureStatus = $adapter->verifySignature($request, $provider);

    if ($signatureStatus->isInvalid()) {
        $this->callbackEvents->recordRejected($provider, $request, 'INVALID_SIGNATURE');
        return response()->json(['result' => 'rejected'], 403);
    }

    $parsed = $adapter->parseCallback($request);
    $idempotencyKey = $adapter->buildIdempotencyKey($parsed);
}

```

```

$result = DB::transaction(function () use ($provider, $request, $parsed, $idempotencyKey) {
    $event = $this->callbackEvents->recordParsed($provider, $request, $parsed, $idempotencyKey);
    return $this->gatewayTransactions->upsertFromCallback($provider, $event, $parsed);
});

return response()->json([
    'result' => 'accepted',
    'gatewayTransactionId' => $result->gateway_txn_id
], 200);
}

```

3.2 Normalized Callback Object

Every provider adapter must output this shape:

```

{
    "providerCode": "MPESA_KE",
    "providerTransactionId": "QL91ABCD",
    "paymentReference": "ACC-900045",
    "payerMsisdn": "254700111222",
    "amount": 2500.00,
    "currency": "KES",
    "paidAt": "2026-05-29T09:42:00Z",
    "channelHint": "MPESA_PAYBILL",
    "rawProviderStatus": "SUCCESS"
}

```

4. Step 3 - Store Callback Event

Insert one row into gateway_callback_event.

Sample row:

```

insert into gateway_callback_event (
    callback_event_id,
    provider_id,
    provider_event_id,
    provider_transaction_id,
    idempotency_key,
    received_at,
    signature_status,
    parse_status,
    http_status_returned,
    raw_payload_redacted,
    created_at
) values (
    'cbe-20260529-000001',
    'prv-mpesa-ke',
    null,
    'QL91ABCD',
    'MPESA_KE:QL91ABCD',
    now(),
    'VALID',
    'PARSED',
    200,
    '{"TransID":"QL91ABCD","TransAmount":"2500.00","BillRefNumber":"ACC-900045","MSISDN":"254700***"}',
    now()
);

```

Meaning:

- provider_transaction_id is the provider's payment id.
- idempotency_key prevents duplicate callback processing.
- raw_payload_redacted helps support without leaking sensitive fields.

5. Step 4 - Upsert Gateway Transaction

The same callback transaction creates or updates one row in gateway_transaction.

5.1 First Callback

```

insert into gateway_transaction (
    gateway_txn_id,
    operator_code,

```

```

    provider_id,
    channel_id,
    provider_transaction_id,
    payment_reference,
    payer_msisdn,
    amount,
    currency_code,
    paid_at,
    status,
    target_type,
    created_at,
    updated_at
) values (
    'gtx-20260529-000001',
    'WIK',
    'prv-mpesa-ke',
    'chn-mpesa-paybill',
    'QL91ABCD',
    'ACC-900045',
    '254700111222',
    2500.00,
    'KES',
    '2026-05-29T09:42:00Z',
    'ROUTING_PENDING',
    'UNKNOWN',
    now(),
    now()
);

```

5.2 Duplicate Callback

If another callback arrives with:

```

provider_code = MPESA_KE
provider_transaction_id = QL91ABCD
amount = 2500.00
currency = KES
paid_at = 2026-05-29T09:42:00Z

```

Then do not insert a second gateway transaction and do not call PAY-01 or BIL-05 again. Return 200 to the provider.

5.3 Duplicate Conflict

If another callback arrives with same provider transaction id but different amount:

```

existing amount = 2500.00
new amount = 3000.00

```

Set the existing transaction or a conflict record to:

```

status = DUPLICATE_CONFLICT
hold_reason = DUPLICATE_CONFLICT

```

Create a gateway_pending_resolution row so operations can investigate.

6. Step 5 - Resolve Target

The routing worker loads transactions with:

```

select *
from gateway_transaction
where status in ('ROUTING_PENDING', 'ROUTING_RETRY')
order by paid_at asc
limit 100;

```

For each transaction, it resolves the payment target.

6.1 Reference Is An Account Or Invoice

If payment_reference = ACC-900045, call the customer/account lookup API or internal repository and resolve:

```

{
  "targetType": "ACCOUNT",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "invoiceId": null,
  "applicationModule": "BIL-01-PAY-01"
}

```

The routing worker will call PAY-01.

6.2 Reference Is A Wallet Alias

If the channel is configured as WALLET_TOPUP or the reference resolves to a wallet alias, resolve:

```
{
  "targetType": "WALLET",
  "customerId": "CUS-100045",
  "subscriptionId": "SUB-700045",
  "walletId": "WAL-200045",
  "applicationModule": "BIL-05"
}
```

The routing worker will call BIL-05.

6.3 Reference Cannot Be Resolved

If no account, invoice, customer, or wallet is found:

```
update gateway_transaction
set status = 'HELD',
    hold_reason = 'UNKNOWN_REFERENCE',
    updated_at = now()
where gateway_txn_id = 'gtx-20260529-000001';
```

Then insert:

```
insert into gateway_pending_resolution (
  pending_id,
  gateway_txn_id,
  reason_code,
  status,
  created_at
) values (
  'gpr-20260529-000001',
  'gtx-20260529-000001',
  'UNKNOWN_REFERENCE',
  'OPEN',
  now()
);
```

Emit `sophix.paygw.payment.held`.

7. Step 6 - Route To PAY-01 Payment Application

If the target is invoice/account payment, the routing worker calls:

```
POST /api/payments/receive
Authorization: Bearer <internal-service-token>
Idempotency-Key: PAYGW:MPESA_KE:QL91ABCD
Content-Type: application/json
```

Request body:

```
{
  "operatorCode": "WIK",
  "customerId": "CUS-100045",
  "accountId": "ACC-900045",
  "invoiceId": null,
  "paymentReference": "MPESA_KE:QL91ABCD",
  "paidAmount": 2500.00,
  "currency": "KES",
  "paymentMethod": "MOBILE_MONEY",
  "gatewayProviderCode": "MPESA_KE",
  "gatewayTransactionId": "gtx-20260529-000001",
  "paidAt": "2026-05-29T09:42:00Z"
}
```

Expected success response from PAY-01:

```
{
  "paymentId": "PAY-20260529-000001",
  "status": "APPLIED",
  "allocatedAmount": 2500.00,
  "unallocatedAmount": 0.00
}
```

Then PAY-GW updates:

```

update gateway_transaction
set status = 'APPLIED',
    target_type = 'ACCOUNT',
    target_id = 'ACC-900045',
    customer_id = 'CUS-100045',
    account_id = 'ACC-900045',
    application_module = 'BIL-01-PAY-01',
    application_reference_id = 'PAY-20260529-000001',
    updated_at = now()
where gateway_txn_id = 'gtx-20260529-000001';

```

And emits:

```

{
  "eventType": "GatewayPaymentApplied",
  "gatewayTxnId": "gtx-20260529-000001",
  "applicationModule": "BIL-01-PAY-01",
  "applicationReferenceId": "PAY-20260529-000001",
  "customerId": "CUS-100045",
  "amount": 2500.00,
  "currency": "KES"
}

```

8. Step 7 - Route To BIL-05 Wallet Topup

If the target is a prepaid wallet, the routing worker calls:

```

POST /api/wallets/WAL-200045/credit
Authorization: Bearer <internal-service-token>
Idempotency-Key: PAYGW:MPESA_KE:QL91EFGH
Content-Type: application/json

```

Request body:

```

{
  "operatorCode": "WIK",
  "walletId": "WAL-200045",
  "customerId": "CUS-100045",
  "subscriptionId": "SUB-700045",
  "amount": 1000.00,
  "currency": "KES",
  "creditType": "EXTERNAL_TOPUP",
  "gatewayProviderCode": "MPESA_KE",
  "gatewayReference": "MPESA_KE:QL91EFGH",
  "gatewayTransactionId": "gtx-20260529-000002",
  "paidAt": "2026-05-29T09:45:00Z"
}

```

Expected success response from BIL-05:

```

{
  "walletTransactionId": "WTX-20260529-000001",
  "walletId": "WAL-200045",
  "newBalance": 1800.00,
  "status": "POSTED"
}

```

Then PAY-GW updates:

```

update gateway_transaction
set status = 'APPLIED',
    target_type = 'WALLET',
    target_id = 'WAL-200045',
    customer_id = 'CUS-100045',
    subscription_id = 'SUB-700045',
    wallet_id = 'WAL-200045',
    application_module = 'BIL-05',
    application_reference_id = 'WTX-20260529-000001',
    updated_at = now()
where gateway_txn_id = 'gtx-20260529-000002';

```

9. Step 8 - Record Routing Attempts

Every internal call to PAY-01 or BIL-05 must be recorded in gateway_routing_attempt.

Success example:

```

insert into gateway_routing_attempt (

```

```

    attempt_id,
    gateway_txn_id,
    attempt_no,
    target_module,
    target_endpoint,
    status,
    request_payload_redacted,
    response_status,
    response_payload_redacted,
    created_at
) values (
    'gra-20260529-000001',
    'gtx-20260529-000001',
    1,
    'BIL-01-PAY-01',
    '/api/payments/receive',
    'SUCCESS',
    '{"paidAmount":2500.00,"paymentReference":"MPESA_KE:QL91ABCD"}',
    201,
    '{"paymentId":"PAY-20260529-000001"}',
    now()
);

```

Retryable failure example:

```

insert into gateway_routing_attempt (
    attempt_id,
    gateway_txn_id,
    attempt_no,
    target_module,
    target_endpoint,
    status,
    response_status,
    error_code,
    created_at
) values (
    'gra-20260529-000002',
    'gtx-20260529-000001',
    2,
    'BIL-01-PAY-01',
    '/api/payments/receive',
    'FAILED_RETRYABLE',
    503,
    'TARGET_UNAVAILABLE',
    now()
);

```

Then set transaction status:

```

update gateway_transaction
set status = 'ROUTING_RETRY',
    last_error_code = 'TARGET_UNAVAILABLE',
    updated_at = now()
where gateway_txn_id = 'gtx-20260529-000001';

```

10. Step 9 - Handle Provider Callback Responses

The provider response depends on what happened before durable storage.

Scenario	Store callback?	Route payment?	HTTP response
Valid callback, stored	Yes	Async	200
Duplicate callback, same values	Yes	No	200
Duplicate callback, conflicting values	Yes	No, hold	200 or provider-specific accepted response
Invalid signature	Minimal audit only	No	403
Malformed payload	Audit if parseable	No	400
Database unavailable before store	No	No	500 so provider retries

Important: once money appears real and the callback is stored, prefer holding and repairing over rejecting. Rejecting a real payment can make provider and customer support more difficult.

11. Step 10 - Operations Repairs Held Payment

A Backoffice user opens:

```
GET /api/payment-gateways/pending-resolutions?status=OPEN
```

Example response:

```
{
  "items": [
    {
      "pendingId": "gpr-20260529-000001",
      "gatewayTxnId": "gtx-20260529-000003",
      "reasonCode": "UNKNOWN_REFERENCE",
      "providerCode": "MPESA_KE",
      "paymentReference": "AC90045",
      "payerMsisdnMasked": "254700***222",
      "amount": 1000.00,
      "currency": "KES",
      "paidAt": "2026-05-29T10:00:00Z"
    }
  ]
}
```

If operations identifies the correct account, it calls:

```
POST /api/payment-gateways/pending-resolutions/gpr-20260529-000001/route-to-payment
Content-Type: application/json
```

```
{
  "accountId": "ACC-900045",
  "customerId": "CUS-100045",
  "reason": "Customer entered account reference without dash."
}
```

PAY-GW then:

1. Updates the gateway_transaction target fields.
2. Marks pending resolution as RESOLVED.
3. Sets transaction status to ROUTING_PENDING.
4. Routing worker sends it to PAY-01.

Do not let the repair API directly mutate invoice or wallet balances.

12. Step 11 - Run Settlement Reconciliation

The reconciliation worker runs daily per provider after the provider settlement window closes.

It loads provider settlement rows from API, SFTP file, or manual upload.

Example normalized settlement row:

```
{
  "providerCode": "MPESA_KE",
  "settlementDate": "2026-05-29",
  "providerTransactionId": "QL91ABCD",
  "amount": 2500.00,
  "currency": "KES",
  "paidAt": "2026-05-29T09:42:00Z",
  "settlementStatus": "SUCCESS"
}
```

12.1 Reconciliation Logic

For each provider settlement row:

1. Find gateway_transaction by (provider_id, provider_transaction_id).
2. If not found, create MISSING_IN_BSS.
3. If found but amount differs, create AMOUNT_MISMATCH.
4. If found but status is not APPLIED, create STATUS_MISMATCH.
5. If found and values match, mark transaction RECONCILED.

Then compare SOPHIX gateway transactions for the same provider/date that were not present in provider settlement. Create MISSING_IN_PROVIDER when needed.

12.2 Create Run

```
insert into gateway_reconciliation_run (
  run_id,
  provider_id,
  settlement_date,
  status,
  provider_total_amount,
  sophix_total_amount,
  provider_txn_count,
  sophix_txn_count,
  discrepancy_count,
  started_at
) values (
  'rec-20260530-mpesa',
  'prv-mpesa-ke',
  '2026-05-29',
  'RUNNING',
  0,
  0,
  0,
  0,
  0,
  now()
);
```

12.3 Create Discrepancy Item

```
insert into gateway_reconciliation_item (
  item_id,
  run_id,
  provider_transaction_id,
  gateway_txn_id,
  discrepancy_type,
  provider_amount,
  sophix_amount,
  status,
  created_at
) values (
  'rci-20260530-000001',
  'rec-20260530-mpesa',
  'QL91ZZZ',
  null,
  'MISSING_IN_BSS',
  1000.00,
  null,
  'OPEN',
  now()
);
```

13. Step 12 - Reversal Or Chargeback Event

Some providers may send reversal, chargeback, or failed settlement events.

PAY-GW receives the event and finds the original gateway transaction.

Do not directly reverse invoices or wallet ledgers in PAY-GW.

Recommended handling:

Original application module	Action
BIL-01-PAY-01	Call PAY-01 reversal API if policy allows automatic reversal. Otherwise create approval/repair task.
BIL-05	Call BIL-05 debit adjustment or reversal endpoint if configured. Otherwise create approval/repair task.
Unknown/not applied	Mark gateway transaction REVERSED or REJECTED as appropriate.

Initial implementation may hold all provider reversals for finance review. That is safer than automatic ledger reversal without business approval.

14. Step 13 - Frontend Reads Payment Channels

Self-care and sales apps should call:

```
GET /api/payment-gateways/channels?operatorCode=WIK&status=ACTIVE&context=SELF_CARE_PAYMENT
```

Example response:

```
{
  "items": [
    {
      "channelCode": "MPESA_PAYBILL",
      "displayName": "M-Pesa Paybill",
      "paymentMethod": "MOBILE_MONEY",
      "currency": "KES",
      "instructions": {
        "en": "Use Paybill 123456 and enter your customer account number as the reference.",
        "sw": "Tumia Paybill 123456 na uweke nambari yako ya akaunti kama rejeleo."
      }
    }
  ]
}
```

The frontend renders this text. It does not build provider instructions itself.

15. Validation Rules

Implement these checks before marking a transaction routeable:

Check	Failure status
Provider is active	REJECTED
Channel is active	HELD with CHANNEL_INACTIVE
Signature is valid	REJECTED
Provider transaction id is present	REJECTED
Amount is positive	REJECTED
Currency matches channel/provider	HELD with CURRENCY_MISMATCH
Paid timestamp is valid	HELD with INVALID_PAID_AT
Idempotency conflict absent	DUPLICATE_CONFLICT
Target reference resolves	ROUTING_PENDING, otherwise HELD

16. Error Codes

Use stable error codes so QA and Backoffice can handle failures consistently.

Error code	Meaning
PAYGW_PROVIDER_INACTIVE	Provider is not active.
PAYGW_SIGNATURE_INVALID	Signature or HMAC verification failed.
PAYGW_PAYLOAD_MALFORMED	Provider payload cannot be parsed.
PAYGW_DUPLICATE_CONFLICT	Same provider transaction id has conflicting values.
PAYGW_UNKNOWN_REFERENCE	Payment reference cannot be resolved.
PAYGW_MULTIPLE_TARGETS	Payment reference maps to multiple targets.
PAYGW_TARGET_UNAVAILABLE	Internal target module is down or timed out.
PAYGW_TARGET_REJECTED	PAY-01 or BIL-05 rejected the request.
PAYGW_RECON_MISSING_IN_BSS	Provider settlement has a row missing in SOPHIX.
PAYGW_RECON_AMOUNT_MISMATCH	Provider and SOPHIX amounts differ.

17. Suggested Worker Commands

These are Laravel command or scheduler jobs. They are not separate microservices unless deployment later chooses to split them.

```
paygw:route-pending
```

```
paygw:retry-routing
paygw:reconcile-provider --provider=MPESA_KE --date=2026-05-29
paygw:expire-stale-held
paygw:replay-callback --callback-event-id=cbe-20260529-000001
```

Recommended schedule:

Command	Frequency
paygw:route-pending	Every minute, or continuous queue worker.
paygw:retry-routing	Every 5 minutes.
paygw:reconcile-provider	Daily after provider settlement cut-off.
paygw:expire-stale-held	Daily.
paygw:replay-callback	Manual only.

Workers can be consolidated into one queue worker process that handles multiple job types.

18. Minimum Implementation Order

Build in this order:

1. Database migrations for provider, channel, callback event, transaction, routing attempt, reconciliation, pending resolution.
2. Provider adapter interface.
3. M-Pesa-style adapter with parser, signature stub, redaction, idempotency key builder.
4. Public callback controller.
5. Gateway transaction repository with duplicate and conflict handling.
6. Routing resolver.
7. PAY-01 routing client.
8. BIL-05 routing client.
9. Routing worker and retry logic.
10. Held payment operations APIs.
11. Reconciliation run and item tables.
12. Daily reconciliation worker.
13. Channel configuration API for frontend.
14. RBAC permissions and audit log.
15. Metrics and alerts.

This order lets QA test real callback-to-payment behavior before adding all reconciliation features.

19. Test Scenarios

19.1 Valid POSTPAID Payment

Input:

- Callback amount 2500.00.
- Reference ACC-900045.
- Target account exists.

Expected:

- Callback event stored.
- Gateway transaction becomes APPLIED.
- PAY-01 receives one POST /api/payments/receive.

- PAY-01 emits its normal payment applied event.
- PAY-GW emits GatewayPaymentApplied.

19.2 Valid Wallet Topup

Input:

- Callback amount 1000.00.
- Reference resolves to WAL-200045.

Expected:

- Callback event stored.
- Gateway transaction becomes APPLIED.
- BIL-05 receives one wallet credit request.
- Wallet balance increases.
- Tax module later consumes BIL-05 wallet topup event, not PAY-GW event.

19.3 Duplicate Callback

Input:

- Same provider transaction id sent twice with same amount and currency.

Expected:

- One gateway transaction.
- One PAY-01 or BIL-05 call.
- Second provider callback gets 200.

19.4 Duplicate Conflict

Input:

- Same provider transaction id sent twice with different amount.

Expected:

- Transaction status DUPLICATE_CONFLICT.
- Pending resolution row created.
- No target module call for conflicting callback.

19.5 Unknown Reference

Input:

- Valid payment reference not found in customer/account/wallet lookup.

Expected:

- Gateway transaction status HELD.
- Reason UNKNOWN_REFERENCE.
- Backoffice repair API can route it later.

19.6 Target Module Down

Input:

- Valid account payment.
- PAY-01 returns 503.

Expected:

- Routing attempt saved as FAILED_RETRYABLE.

- Transaction status ROUTING_RETRY.
- Retry worker attempts later with same idempotency key.

19.7 Reconciliation Missing In BSS

Input:

- Provider settlement file contains transaction QL91ZZZZ.
- No gateway transaction exists.

Expected:

- Reconciliation item MISSING_IN_BSS.
- Backoffice shows discrepancy.

20. Developer Checklist

Before marking implementation complete:

- Provider secrets are loaded from secret manager.
- Callback endpoint stores before acknowledging provider.
- Callback event redaction is tested.
- Duplicate callback does not double-credit.
- Duplicate conflict creates held resolution.
- PAY-01 route uses idempotency key.
- BIL-05 route uses idempotency key.
- Routing attempts are recorded for every target call.
- Held transaction repair is audited.
- Reconciliation can create discrepancy items.
- FE apps can read active channel instructions.
- Metrics exist for callback success/failure, routing backlog, held queue, and reconciliation discrepancies.